

Metadata Journaling on VSFS

Tasks

- Disk image + VSFS-like layout you will modify
- Redo journal format (multi-transaction)
- Command semantics: create / install
- Validator-driven correctness

Core idea

Describe changes → Commit → Install

create: append DATA records



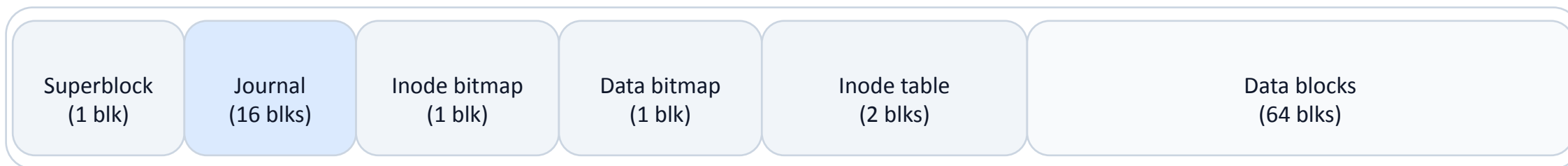
append COMMIT



install: replay committed DATA

Provided disk image layout (VSFS-like)

You will be given an image with a fixed layout:



Parameters (fixed)

- Block size = 4096 bytes
- Inode size = 128 bytes
- Journal lives in 16 blocks (append-only)
- You modify the image in-place; you do not build mkfs

What the validator checks

- Bitmaps agree with actual inodes/blocks in use
- Directory entries point to allocated inodes
- Inode block pointers point to allocated data blocks
- No double-allocations or dangling references

Core on-disk structures (simplified for the lab)

Inode (128 bytes)

- type (free/file/dir), links, size
- direct block pointers (small files)
- optional: timestamps (only if validator cares)

Directory entry (fixed-size)

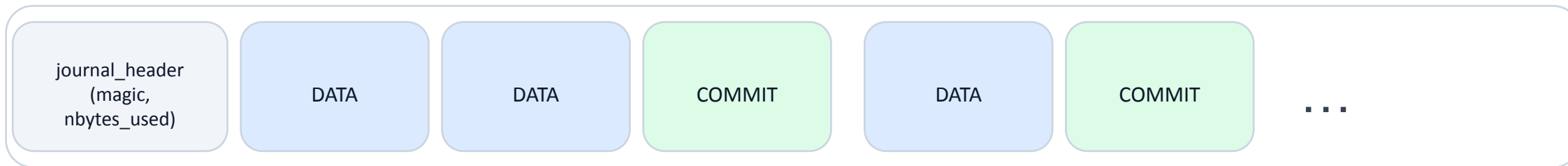
- inode number + fixed-length name
- unused slot: inode == 0
- directory block = array of dirents

Invariants you must preserve

- Dirent → inode must reference an allocated inode
- Allocated inode must have a valid type and bounds-checked pointers
- Inode pointers → allocated data blocks (if any)
- Bitmaps reflect the above (no leaks, no doubles)

Journal block format (16 blocks long, many transactions)

Journal is an append-only redo log inside 1 block:



Record types

- Every record begins with `rec_header { type, size }`
- DATA record: `{ block_no, full 4096B block image }`
- COMMIT record: seals the current transaction (no payload)
- Parsing = read `rec_header`, interpret by type, jump by size

Transaction rule

- DATA records since last COMMIT belong to current txn
- Txn is valid only if its COMMIT is present and fully readable
- Tail DATA without COMMIT is discarded by install
- Journal fills → require install (or refuse new txn)

Command semantics (what your program must do)

create <name>

- Read current metadata (bitmaps, inode table, directory blocks)
- Compute updated versions of affected metadata blocks in memory
- Append DATA records for each modified metadata block
- Append COMMIT record to the journal
- Do NOT write metadata to home locations here

install

- Scan journal in order, grouping DATA records until COMMIT
- For each committed txn: write each DATA block image to its block_no
- If an incomplete txn exists (no COMMIT): do not apply; discard tail
- Clear the journal (reset nbytes_used) after installation

If journal is full

Refuse new create
(or require install first).

Testing approach

- Run sequences of create to fill multiple transactions
- Run install sequences
- Goal: validator passes for all tested points

Deliverables

- journal.c (create, install)

Common failure modes (what validator catches)

- Dirent points to a free inode (dangling entry)
- Inode allocated but unreachable (leak)
- Data bitmap says “free” but inode points to it (mismatch)
- Double allocation (two inodes share a block)
- Journal tail corruption not safely ignored